`http.server` — HTTP servers

Source code: Lib/http/server.py

This module defines classes for implementing HTTP servers.

> **Warning:** `http.server` is not recommended for production. It only implements basic security checks.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See WebAssembly platforms for more information.

One class, `HTTPServer`, is a `socketserver.TCPServer` subclass. It creates and listens at the HTTP socket, dispatching the requests to a handler. Code to create and run the server looks like this:

```
def run(server_class=HTTPServer, handler_class=BaseHTTPRequestHandler):
    server_address = ('', 8000)
    httpd = server_class(server_address, handler_class)
    httpd.serve_forever()
```

class http.server.**HTTPServer**(*server_address*, *RequestHandlerClass*)

This class builds on the `TCPServer` class by storing the server address as instance variables named `server_name` and `server_port`. The server is accessible by the handler, typically through the handler's `server` instance variable.

class http.server.**ThreadingHTTPServer**(*server_address*, *RequestHandlerClass*)

This class is identical to HTTPServer but uses threads to handle requests by using the `ThreadingMixIn`. This is useful to handle web browsers pre-opening sockets, on which `HTTPServer` would wait indefinitely.

> *Added in version 3.7.*

class http.server.**HTTPSServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*, ***, *certfile*, *keyfile=None*, *password=None*, *alpn_protocols=None*)

Subclass of `HTTPServer` with a wrapped socket using the `ssl` module. If the `ssl` module is not available, instantiating a HTTPSServer object fails with a `RuntimeError`.

The *certfile* argument is the path to the SSL certificate chain file, and the *keyfile* is the path to file containing the private key.

A *password* can be specified for files protected and wrapped with PKCS#8, but beware that this could possibly expose hardcoded passwords in clear.

> **See also:** See `ssl.SSLContext.load_cert_chain()` for additional information on the ac-

cepted values for *certfile*, *keyfile* and *password*.

When specified, the *alpn_protocols* argument must be a sequence of strings specifying the "Application-Layer Protocol Negotiation" (ALPN) protocols supported by the server. ALPN allows the server and the client to negotiate the application protocol during the TLS handshake.

By default, it is set to `["http/1.1"]`, meaning the server supports HTTP/1.1.

> *Added in version 3.14.*

class http.server.**ThreadingHTTPSServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*, ***, *certfile*, *keyfile=None*, *password=None*, *alpn_protocols=None*)

This class is identical to `HTTPSServer` but uses threads to handle requests by inheriting from `ThreadingMixIn`. This is analogous to `ThreadingHTTPServer` only using `HTTPSServer`.

> *Added in version 3.14.*

The `HTTPServer`, `ThreadingHTTPServer`, `HTTPSServer` and `ThreadingHTTPSServer` must be given a *RequestHandlerClass* on instantiation, of which this module provides three different variants:

class http.server.**BaseHTTPRequestHandler**(*request*, *client_address*, *server*)

This class is used to handle the HTTP requests that arrive at the server. By itself, it cannot respond to any actual HTTP requests; it must be subclassed to handle each request method (for example, `'GET'` or `'POST'`). `BaseHTTPRequestHandler` provides a number of class and instance variables, and methods for use by subclasses.

The handler will parse the request and the headers, then call a method specific to the request type. The method name is constructed from the request. For example, for the request method `SPAM`, the `do_SPAM()` method will be called with no arguments. All of the relevant information is stored in instance variables of the handler. Subclasses should not need to override or extend the `__init__()` method.

`BaseHTTPRequestHandler` has the following instance variables:

client_address

Contains a tuple of the form `(host, port)` referring to the client's address.

server

Contains the server instance.

close_connection

Boolean that should be set before `handle_one_request()` returns, indicating if another request may be expected, or if the connection should be shut down.

requestline

Contains the string representation of the HTTP request line. The terminating CRLF is stripped. This attribute should be set by `handle_one_request()`. If no valid request line was processed, it should be set to the empty string.

command

Contains the command (request type). For example, `'GET'`.

path

Contains the request path. If query component of the URL is present, then `path` includes the query. Using the terminology of **RFC 3986**, `path` here includes `hier-part` and the `query`.

request_version

Contains the version string from the request. For example, `'HTTP/1.0'`.

headers

Holds an instance of the class specified by the `MessageClass` class variable. This instance parses and manages the headers in the HTTP request. The `parse_headers()` function from `http.client` is used to parse the headers and it requires that the HTTP request provide a valid **RFC 5322** style header.

rfile

An `io.BufferedIOBase` input stream, ready to read from the start of the optional input data.

wfile

Contains the output stream for writing a response back to the client. Proper adherence to the HTTP protocol must be used when writing to this stream in order to achieve successful inter-operation with HTTP clients.

> *Changed in version 3.6:* This is an `io.BufferedIOBase` stream.

`BaseHTTPRequestHandler` has the following attributes:

server_version

Specifies the server software version. You may want to override this. The format is multiple whitespace-separated strings, where each string is of the form name[/version]. For example, `'BaseHTTP/0.2'`.

sys_version

Contains the Python system version, in a form usable by the `version_string` method and the `server_version` class variable. For example, `'Python/1.4'`.

error_message_format

Specifies a format string that should be used by `send_error()` method for building an error response to the client. The string is filled by default with variables from `responses` based on the status code that passed to `send_error()`.

error_content_type

Specifies the Content-Type HTTP header of error responses sent to the client. The default value is `'text/html'`.

protocol_version

Specifies the HTTP version to which the server is conformant. It is sent in responses to let the client know the server's communication capabilities for future requests. If set to `'HTTP/1.1'`, the server will permit HTTP persistent connections; however, your server *must* then include an accurate `Content-Length` header (using `send_header()`) in all of its responses to clients. For backwards compatibility, the setting defaults to `'HTTP/1.0'`.

`MessageClass`

Specifies an `email.message.Message`-like class to parse HTTP headers. Typically, this is not overridden, and it defaults to `http.client.HTTPMessage`.

`responses`

This attribute contains a mapping of error code integers to two-element tuples containing a short and long message. For example, `{code: (shortmessage, longmessage)}`. The *shortmessage* is usually used as the *message* key in an error response, and *longmessage* as the *explain* key. It is used by `send_response_only()` and `send_error()` methods.

A `BaseHTTPRequestHandler` instance has the following methods:

`handle()`

Calls `handle_one_request()` once (or, if persistent connections are enabled, multiple times) to handle incoming HTTP requests. You should never need to override it; instead, implement appropriate `do_*()` methods.

`handle_one_request()`

This method will parse and dispatch the request to the appropriate `do_*()` method. You should never need to override it.

`handle_expect_100()`

When an HTTP/1.1 conformant server receives an `Expect: 100-continue` request header it responds back with a `100 Continue` followed by `200 OK` headers. This method can be overridden to raise an error if the server does not want the client to continue. For example, the server can choose to send `417 Expectation Failed` as a response header and `return False`.

> *Added in version 3.2.*

`send_error(code, message=None, explain=None)`

Sends and logs a complete error reply to the client. The numeric *code* specifies the HTTP error code, with *message* as an optional, short, human readable description of the error. The *explain* argument can be used to provide more detailed information about the error; it will be formatted using the `error_message_format` attribute and emitted, after a complete set of headers, as the response body. The `responses` attribute holds the default values for *message* and *explain* that will be used if no value is provided; for unknown codes the default value for both is the string `???`. The body will be empty if the method is HEAD or the response code is one of the following: `1xx`, `204 No Content`, `205 Reset Content`, `304 Not Modified`.

> *Changed in version 3.4:* The error response includes a Content-Length header. Added the *explain* argument.

send_response(*code*, *message=None*)

Adds a response header to the headers buffer and logs the accepted request. The HTTP response line is written to the internal buffer, followed by *Server* and *Date* headers. The values for these two headers are picked up from the `version_string()` and `date_time_string()` methods, respectively. If the server does not intend to send any other headers using the `send_header()` method, then `send_response()` should be followed by an `end_headers()` call.

> *Changed in version 3.3:* Headers are stored to an internal buffer and `end_headers()` needs to be called explicitly.

send_header(*keyword*, *value*)

Adds the HTTP header to an internal buffer which will be written to the output stream when either `end_headers()` or `flush_headers()` is invoked. *keyword* should specify the header keyword, with *value* specifying its value. Note that, after the send_header calls are done, `end_headers()` MUST BE called in order to complete the operation.

This method does not reject input containing CRLF sequences.

> *Changed in version 3.2:* Headers are stored in an internal buffer.

send_response_only(*code*, *message=None*)

Sends the response header only, used for the purposes when `100 Continue` response is sent by the server to the client. The headers not buffered and sent directly the output stream.If the *message* is not specified, the HTTP message corresponding the response *code* is sent.

This method does not reject *message* containing CRLF sequences.

> *Added in version 3.2.*

end_headers()

Adds a blank line (indicating the end of the HTTP headers in the response) to the headers buffer and calls `flush_headers()`.

> *Changed in version 3.2:* The buffered headers are written to the output stream.

flush_headers()

Finally send the headers to the output stream and flush the internal headers buffer.

> *Added in version 3.3.*

log_request(*code='-'*, *size='-'*)

Logs an accepted (successful) request. *code* should specify the numeric HTTP code associated with the response. If a size of the response is available, then it should be passed as the

size parameter.

log_error(*...*)

Logs an error when a request cannot be fulfilled. By default, it passes the message to `log_message()`, so it takes the same arguments (*format* and additional values).

log_message(*format*, *...*)

Logs an arbitrary message to `sys.stderr`. This is typically overridden to create custom error logging mechanisms. The *format* argument is a standard printf-style format string, where the additional arguments to `log_message()` are applied as inputs to the formatting. The client ip address and current date and time are prefixed to every message logged.

version_string()

Returns the server software's version string. This is a combination of the `server_version` and `sys_version` attributes.

date_time_string(*timestamp=None*)

Returns the date and time given by *timestamp* (which must be `None` or in the format returned by `time.time()`), formatted for a message header. If *timestamp* is omitted, it uses the current date and time.

The result looks like `'Sun, 06 Nov 1994 08:49:37 GMT'`.

log_date_time_string()

Returns the current date and time, formatted for logging.

address_string()

Returns the client address.

> *Changed in version 3.3:* Previously, a name lookup was performed. To avoid name resolution delays, it now always returns the IP address.

class http.server.**SimpleHTTPRequestHandler**(*request*, *client_address*, *server*, *directory=None*)

This class serves files from the directory *directory* and below, or the current directory if *directory* is not provided, directly mapping the directory structure to HTTP requests.

> *Changed in version 3.7:* Added the *directory* parameter.

> *Changed in version 3.9:* The *directory* parameter accepts a path-like object.

A lot of the work, such as parsing the request, is done by the base class `BaseHTTPRequestHandler`. This class implements the `do_GET()` and `do_HEAD()` functions.

The following are defined as class-level attributes of `SimpleHTTPRequestHandler`:

server_version

This will be `"SimpleHTTP/" + __version__`, where `__version__` is defined at the module

level.

`extensions_map`

A dictionary mapping suffixes into MIME types, contains custom overrides for the default system mappings. The mapping is used case-insensitively, and so should contain only lower-cased keys.

> *Changed in version 3.9:* This dictionary is no longer filled with the default system mappings, but only contains overrides.

The `SimpleHTTPRequestHandler` class defines the following methods:

`do_HEAD()`

This method serves the `'HEAD'` request type: it sends the headers it would send for the equivalent `GET` request. See the `do_GET()` method for a more complete explanation of the possible headers.

`do_GET()`

The request is mapped to a local file by interpreting the request as a path relative to the current working directory.

If the request was mapped to a directory, the directory is checked for a file named `index.html` or `index.htm` (in that order). If found, the file's contents are returned; otherwise a directory listing is generated by calling the `list_directory()` method. This method uses `os.listdir()` to scan the directory, and returns a `404` error response if the `listdir()` fails.

If the request was mapped to a file, it is opened. Any `OSError` exception in opening the requested file is mapped to a `404`, `'File not found'` error. If there was an `'If-Modified-Since'` header in the request, and the file was not modified after this time, a `304`, `'Not Modified'` response is sent. Otherwise, the content type is guessed by calling the `guess_type()` method, which in turn uses the *extensions_map* variable, and the file contents are returned.

A `'Content-type:'` header with the guessed content type is output, followed by a `'Content-Length:'` header with the file's size and a `'Last-Modified:'` header with the file's modification time.

Then follows a blank line signifying the end of the headers, and then the contents of the file are output.

For example usage, see the implementation of the `test` function in Lib/http/server.py.

> *Changed in version 3.7:* Support of the `'If-Modified-Since'` header.

The `SimpleHTTPRequestHandler` class can be used in the following manner in order to create a very basic webserver serving files relative to the current directory:

```
import http.server
```

```
import socketserver

PORT = 8000

Handler = http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print("serving at port", PORT)
    httpd.serve_forever()
```

SimpleHTTPRequestHandler can also be subclassed to enhance behavior, such as using different index file names by overriding the class attribute `index_pages`.

class http.server.**CGIHTTPRequestHandler**(*request*, *client_address*, *server*)

This class is used to serve either files or output of CGI scripts from the current directory and below. Note that mapping HTTP hierarchic structure to local directory structure is exactly as in SimpleHTTPRequestHandler.

> **Note:** CGI scripts run by the `CGIHTTPRequestHandler` class cannot execute redirects (HTTP code 302), because code 200 (script output follows) is sent prior to execution of the CGI script. This pre-empts the status code.

The class will however, run the CGI script, instead of serving it as a file, if it guesses it to be a CGI script. Only directory-based CGI are used — the other common server configuration is to treat special extensions as denoting CGI scripts.

The `do_GET()` and `do_HEAD()` functions are modified to run CGI scripts and serve the output, instead of serving files, if the request leads to somewhere below the `cgi_directories` path.

The `CGIHTTPRequestHandler` defines the following data member:

cgi_directories

This defaults to `['/cgi-bin', '/htbin']` and describes directories to treat as containing CGI scripts.

The `CGIHTTPRequestHandler` defines the following method:

do_POST()

This method serves the `'POST'` request type, only allowed for CGI scripts. Error 501, “Can only POST to CGI scripts”, is output when trying to POST to a non-CGI url.

Note that CGI scripts will be run with UID of user nobody, for security reasons. Problems with the CGI script will be translated to error 403.

> *Deprecated since version 3.13, will be removed in version 3.15:* `CGIHTTPRequestHandler` is being removed in 3.15. CGI has not been considered a good way to do things for well over a decade. This code has been unmaintained for a while now and sees very little practical use. Retaining it could lead to further security considerations.

Command-line interface

`http.server` can also be invoked directly using the `-m` switch of the interpreter. The following example illustrates how to serve files relative to the current directory:

```
python -m http.server [OPTIONS] [port]
```

The following options are accepted:

port

The server listens to port 8000 by default. The default can be overridden by passing the desired port number as an argument:

```
python -m http.server 9000
```

-b, --bind `<address>`

Specifies a specific address to which it should bind. Both IPv4 and IPv6 addresses are supported. By default, the server binds itself to all interfaces. For example, the following command causes the server to bind to localhost only:

```
python -m http.server --bind 127.0.0.1
```

> *Added in version 3.4.*

> *Changed in version 3.8:* Support IPv6 in the `--bind` option.

-d, --directory `<dir>`

Specifies a directory to which it should serve the files. By default, the server uses the current directory. For example, the following command uses a specific directory:

```
python -m http.server --directory /tmp/
```

> *Added in version 3.7.*

-p, --protocol `<version>`

Specifies the HTTP version to which the server is conformant. By default, the server is conformant to HTTP/1.0. For example, the following command runs an HTTP/1.1 conformant server:

```
python -m http.server --protocol HTTP/1.1
```

> *Added in version 3.11.*

--cgi

`CGIHTTPRequestHandler` can be enabled in the command line by passing the `--cgi` option:

```
python -m http.server --cgi
```

> *Deprecated since version 3.13, will be removed in version 3.15:* `http.server` command line `--cgi` support is being removed because `CGIHTTPRequestHandler` is being removed.

Warning: `CGIHTTPRequestHandler` and the `--cgi` command-line option are not intended for use by untrusted clients and may be vulnerable to exploitation. Always use within a secure environment.

--tls-cert

Specifies a TLS certificate chain for HTTPS connections:

```
python -m http.server --tls-cert fullchain.pem
```

> *Added in version 3.14.*

--tls-key

Specifies a private key file for HTTPS connections.

This option requires `--tls-cert` to be specified.

> *Added in version 3.14.*

--tls-password-file

Specifies the password file for password-protected private keys:

```
python -m http.server \
       --tls-cert cert.pem \
       --tls-key key.pem \
       --tls-password-file password.txt
```

This option requires `--tls-cert` to be specified.

> *Added in version 3.14.*

Security considerations

`SimpleHTTPRequestHandler` will follow symbolic links when handling requests, this makes it possible for files outside of the specified directory to be served.

Methods `BaseHTTPRequestHandler.send_header()` and `BaseHTTPRequestHandler.send_response_only()` assume sanitized input and do not perform input validation such as checking for the presence of CRLF sequences. Untrusted input may result in HTTP Header injection attacks.

Earlier versions of Python did not scrub control characters from the log messages emitted to stderr from `python -m http.server` or the default `BaseHTTPRequestHandler` `.log_message` implementation. This could allow remote clients connecting to your server to send nefarious control codes to your terminal.

> *Changed in version 3.12:* Control characters are scrubbed in stderr logs.